Git Version Control

អ្វីទៅជា Git?

Git គឺជា Version Control System (VCS) ដែលជួយឲ្យ developer

- ជួយ developzer track ការផ្លាស់ប្តូរ code រាល់ដំណាក់កាល
- ប្រើទូលំទូលាយក្នុង Web Development និង Software Development
- Git អនុញ្ញាតឲ្យយើងធ្វើការកែ code ដោយមានសុវត្ថិភាព

Git Repository

GitHub

GitLab

ពេលដែលយើងមិនប្រើប្រាស់ Git

បញ្ហាដែលជួបប្រទះ បើមិនប្រើ Git

- កែ code ខុស ហើយមិនអាចត្រឡប់វិញ
- Copy file ច្រើន version (final, final-v2, final-last)
- មិនដឹងថា code ផ្លាស់ប្តូរនៅពេលណា និងដោយនរណា

អត្ថប្រយោជន៍នៅពេលប្រើប្រាស់ Git

អត្ថប្រយោជន៍នៃ Git

- Track code changes ជា History
- ត្រឡប់ទៅ version ចាស់បាន
- ធ្វើការជាក្រុមបានងាយ
- ប្រើជាមូលដ្ឋានសម្រាប់ GitHub / GitLab

ការចាប់ផ្តើមដំបូង (Initializing a Project)

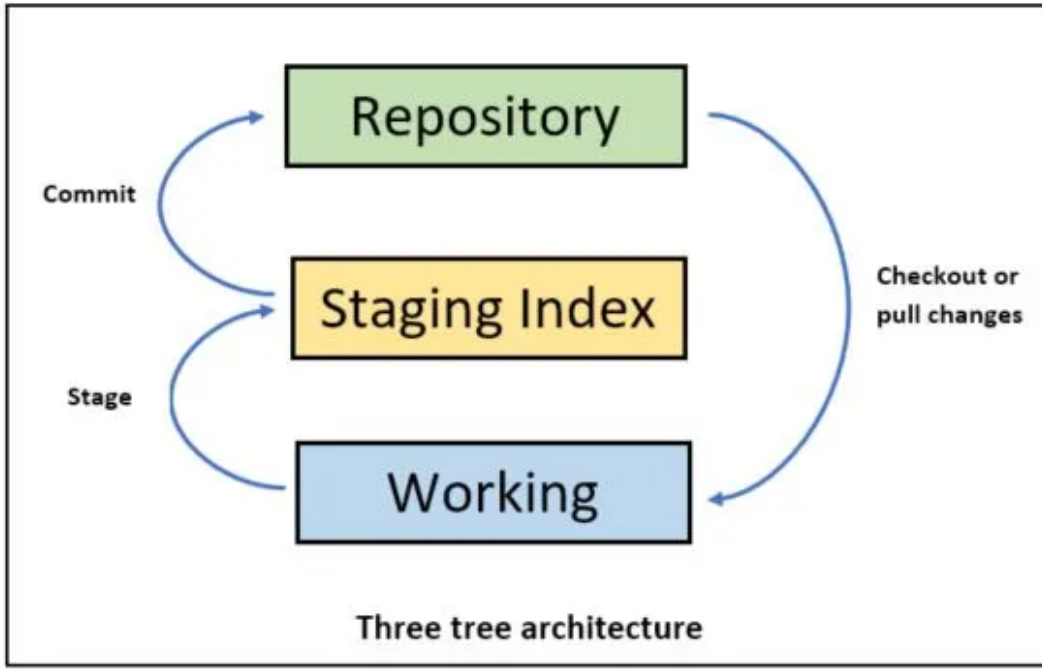
មុននឹងចាប់ផ្តើមប្រើ Git នៅក្នុង Folder គម្រោងរបស់អ្នក អ្នកត្រូវប្រើពាក្យបញ្ជា៖

- git init: បង្កើត Repository ថ្មី (បង្កើត folder លាក់ឈ្មោះ .git)។
- git status: ឆែកមើលថាតើឯកសារណាខ្លះដែល Git កំពុងតាមដាន ឬមិនទាន់បានតាមដាន។

ដំណើរការនៃកូដ (Git Workflow)

ការរក្សាទុកកូដក្នុង Git មាន ៣ ដំណាក់កាលសំខាន់ៃ

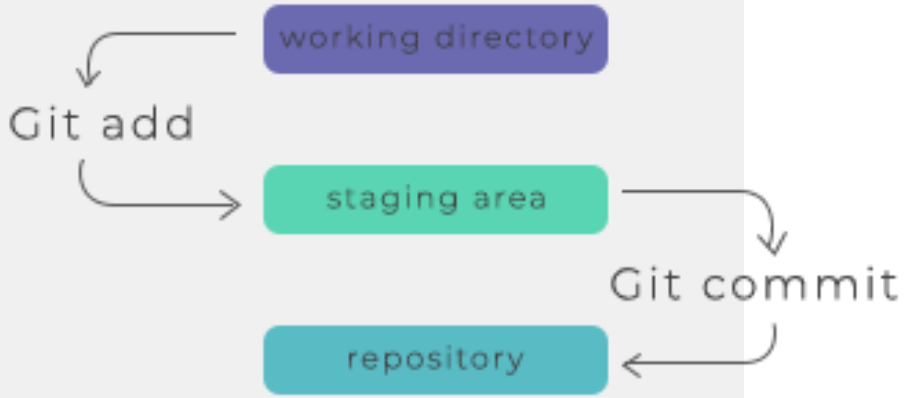
1. Working Directory: កន្លែងដែលអ្នកកំពុងសរសេរកូដជាក់ស្តែង។
2. Staging Area: កន្លែងត្រៀម (ដូចជាការរៀបតវ៉ាន់ដាក់ឡាន)។
3. Repository: កន្លែងរក្សាទុកជាផ្លូវការ (ដូចជាឡានចេញដំណើរ)។



ដំណើរការនៃកូដ (Git Workflow)

នៅពេលអ្នកចង់រក្សាទុកអ្វីដែលអ្នកបានធ្វើ៖

- git add . : បញ្ចូលឯកសារទាំងអស់ទៅកាន់ Staging Area។
- git commit -m "ពន្យល់ពីអ្វីដែលអ្នកបានកែ"



ការប្រើប្រាស់ Git Branch

Branch អនុញ្ញាតឱ្យអ្នកបង្កើតផ្លូវកូដថ្មីដោយមិនប៉ះពាល់ដល់កូដ (Main)៖

- git branch [ឈ្មោះ]: បង្កើតមែកថ្មី។
- git checkout [ឈ្មោះ]: ផ្លាស់ប្តូរទៅកាន់មែកនោះ។
- git merge [ឈ្មោះ]: បញ្ចូលកូដពីមែកតូច មកកាន់មែកមេវិញ។

ការប្រើប្រាស់ជាមួយ Remote (GitHub/GitLab)

ដើម្បីរក្សាទុកកូដនៅលើ Cloud ឬធ្វើការងារជាក្រុម៖

- git remote add origin [URL]: ភ្ជាប់កូដក្នុងម៉ាស៊ីនទៅកាន់ Server។
- git push -u origin main: បញ្ជូនកូដឡើងទៅលើ Server។
- git pull: ទាញយកកូដថ្មីពី Server មកម៉ាស៊ីនវិញ។

តើអ្វីទៅជា .gitignore?

.gitignore គឺជា File ពិសេសមួយដែលប្រើសម្រាប់ប្រាប់ Git ថា 'File ណាខ្លះដែលយើងមិនចង់ឱ្យវា Track ឬបង្ហោះទៅលើ Cloud (ដូចជា GitHub)'។ វាដូចជា 'តម្រង' សម្រាប់ចម្រាញ់យកតែ Code សំខាន់ៗ និងការពារមិនឱ្យឯកសារឥតប្រយោជន៍ ឬឯកសារសម្ងាត់បែកធ្លាយ។"

gitignore

សេចក្តីសន្និដ្ឋាន (Summary)

- git init (ចាប់ផ្តើម)
- git add . (ត្រៀមទុក)
- git add remote (ភ្ជាប់ remote ទៅកាន់ repository
- git commit (រក្សាទុក)
- git push (បញ្ជូនទៅលើ Cloud)
- .gitignore (ហាមឃាត់មិនអោយ File ណាមួយទៅ Server)

តើអ្វីទៅជា Merge Conflict?

Merge Conflict កើតឡើងនៅពេលដែល៖

- មនុស្សពីរនាក់ កែប្រែ បន្ទាត់កូដតែមួយ ក្នុងឯកសារតែមួយ។
- ម្នាក់លុបឯកសារ ចំណែកម្នាក់ទៀតកំពុងកែប្រែឯកសារនោះ។
- Git មិនដឹងថាត្រូវយកកូដរបស់អ្នកណាជាផ្លូវការ ទើបវាទាមទារឱ្យយើងសម្រេចចិត្តដោយផ្ទាល់។

របៀបដែល Git បង្ហាញកូដដែលជាន់គ្នា

នៅពេលមាន Conflict, Git នឹងសរសេរកូដបញ្ចូលគ្នាក្នុងឯកសាររបស់អ្នកក្នុងទម្រង់បែបនេះ៖

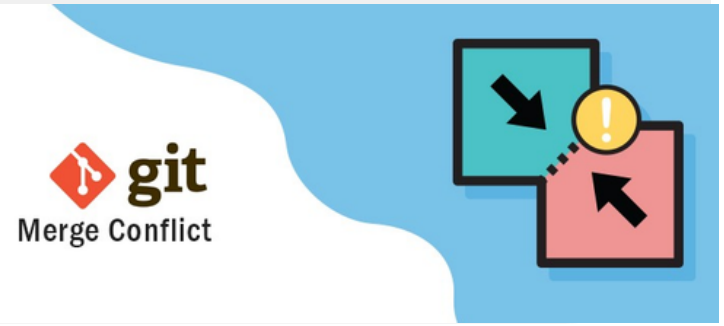
```
<<<<<<< HEAD
```

```
កូដរបស់អ្នក (Current Change)
```

```
=======
```

```
កូដរបស់មិត្តភក្តិ ឬកូដមកពី Branch ផ្សេង (Incoming Change)
```

```
>>>>>>> feature-branch
```



ការពិនិត្យ (Identify)

នៅពេលអ្នក Merge ឬ Pull ហើយឃើញសារប្រកាសអាសន្ន៖

1. ប្រើពាក្យបញ្ជា git status ដើម្បីមើលថាឯកសារណាខ្លះដែលមានជម្លោះ (Unmerged paths)។
2. បើកឯកសារទាំងនោះនៅក្នុង VS Code ឬ Editor របស់អ្នក។

ការបញ្ចប់ (Finalize)

បន្ទាប់ពីកែសម្រួលកូដឱ្យស្អាតរួចរាល់៖

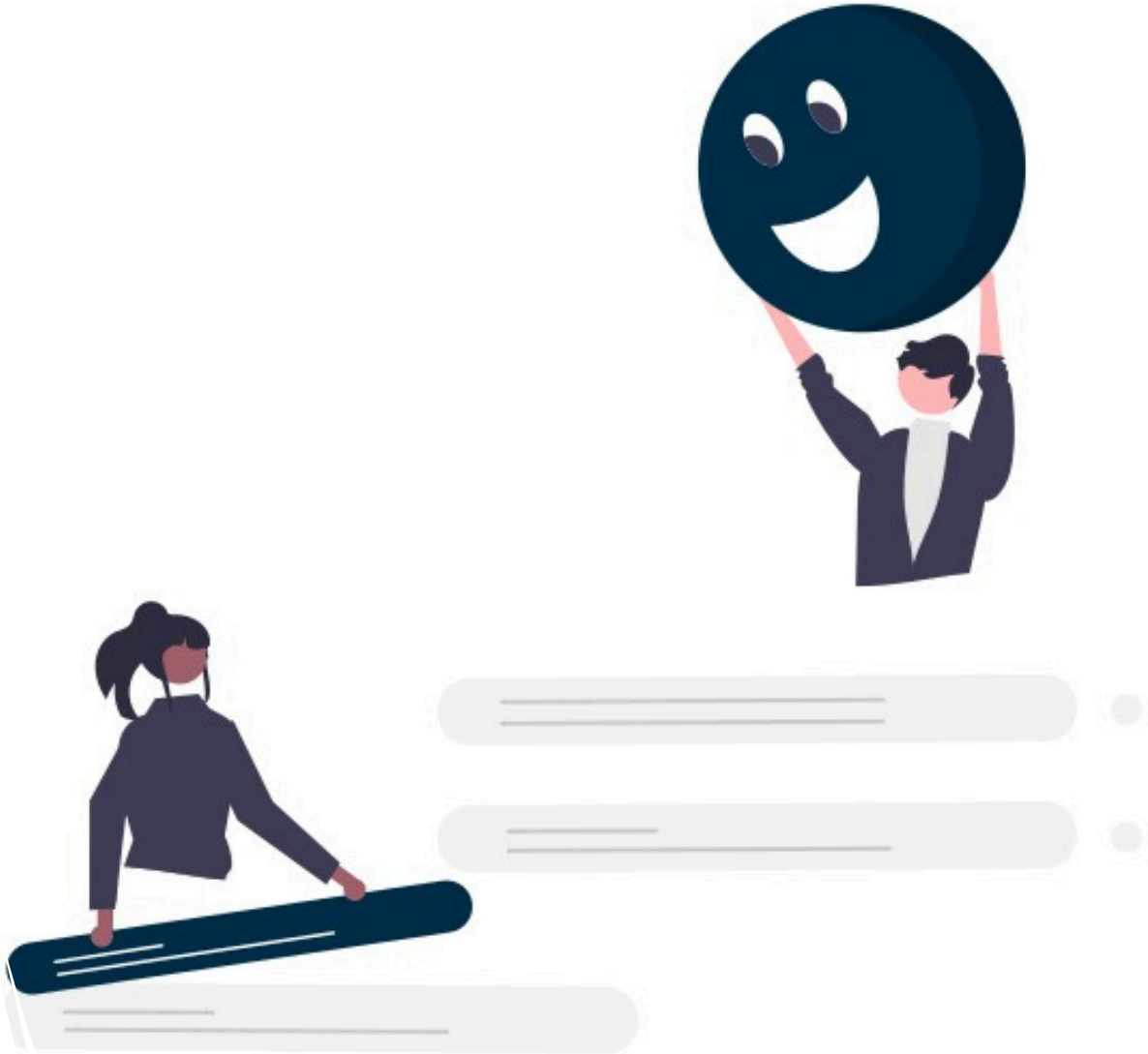
1. ប្រើ git add [ឈ្មោះឯកសារ] ដើម្បីប្រាប់ Git ថាអ្នកបានដោះស្រាយរួចហើយ។
2. ប្រើ git commit -m "Resolved merge conflict in file_name" ដើម្បីបញ្ជាក់ការដោះស្រាយ។

ការជ្រើសរើស (Resolve)

អ្នកមានជម្រើស ៣ ក្នុងការដោះស្រាយ៖

- Accept Current Change: យករក្សាតែកូដរបស់អ្នក។
- Accept Incoming Change: យកកូដដែលទើបនឹងទាញមកថ្មី។
- Accept Both Changes: រក្សាទុកទាំងពីរ (តម្រូវឱ្យអ្នករៀបរយកូដឡើងវិញដោយដៃ)។

គន្លឹះការពារកុំឱ្យជួប Conflict ញឹកញាប់

- Pull ឱ្យបានញឹកញាប់: ដើម្បីទទួលបានកូដថ្មីៗពីក្រុមជានិច្ចៗ
- ធ្វើការលើមែក (Branch) ផ្សេងៗគ្នា: កុំធ្វើការលើ Branch តែមួយជាមួយគ្នាៗ
- Commit ឱ្យបានតូចៗ: ងាយស្រួលដោះស្រាយជាងការបណ្តោយឱ្យកូដជាន់គ្នាច្រើនពេកៗ

Thank you!